

Reviewer Pack — Minimal Order of Affine Generators in Affine Geometry and SA...

Entry fb97e5d749a7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 01:58:06 UTC

Minimal Order of Affine Generators in Affine Geometry and SAT Clause Set Complexity

Entry ID: fb97e5d749a7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 01:58:06 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): SAT Clause Sets

Statement:

For every satisfiability instance with m clauses and n variables, the minimal order of affine generators for its associated algebraic variety is linearly correlated with its clause set complexity $c(\varphi)$, such that $\log(m) = \Theta(\log(n)c(\varphi))$.

Rationale (proposer's reasoning):

Affine geometry provides a structured framework to study the geometric properties of algebraic varieties. This conjecture proposes that the minimal order of affine generators, which measures the complexity of the associated variety, correlates with the clause set complexity of the SAT instance, potentially revealing underlying geometric structures related to computational complexity.

Taxonomy category: AffineGeometry_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bd98f552cca1510c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a satisfiability instance with m clauses and n variables, if the Pearson correlation coefficient (r) between $\log(m)$ and clause set complexity $c(\varphi)$ is ≥ 0.7 for at least 80% of the seeds, then support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - affine geometry AND SAT clause sets - algebraic variety in affine geometry AND clause set complexity - minimal order of affine generators AND related to satisfiability instances

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        m = len(A[0])
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                continue
            for j in range(m):
                A[i][j] /= pivot
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(m):
                        A[j][k] -= factor * A[i][k]
        return A

    def is_sat(instance):
        n, m = len(instance), len(instance[0])
        A = [[0] * (n + 1) for _ in range(n)]
        for i in range(n):
            for j in range(m):
```

```

        if instance[i][j]:
            A[i][j] = 1
            A[i][-1] = 1
    reduced_A = gaussian_elimination(A)
    for row in reduced_A:
        if row[-1] == 1 and all(x == 0 for x in row[:-1]):
            return False
    return True

def clause_set_complexity(instance):
    n, m = len(instance), len(instance[0])
    count = 0
    for i in range(n):
        for j in range(m):
            if instance[i][j]:
                count += 1
    return count

def generate_instance(n, m):
    return [[random.choice([0, 1]) for _ in range(m)] for _ in range(n)]

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    for _ in range(5):
        instance = generate_instance(n, m)
        if not is_sat(instance):
            continue
        c_phi = clause_set_complexity(instance)
        log_n = math.log(n)
        log_m = math.log(m)
        results.append((log_n, log_m, c_phi))

if len(results) < 30:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

n = len(results)
x_sum = sum(x for x, _, _ in results)
y_sum = sum(y for _, y, _ in results)
z_sum = sum(c_phi for _, _, c_phi in results)
xy_sum = sum(x * y for x, y, _ in results)
xz_sum = sum(x * c_phi for x, _, c_phi in results)
yz_sum = sum(y * c_phi for _, y, c_phi in results)

x_mean = x_sum / n
y_mean = y_sum / n
z_mean = z_sum / n

xy_cov = xy_sum - x_mean * y_mean

```

```

xz_cov = xz_sum - x_mean * z_mean
yz_cov = yz_sum - y_mean * z_mean

x_var = sum((x - x_mean) ** 2 for x, _, _ in results) / n
y_var = sum((y - y_mean) ** 2 for _, y, _ in results) / n
z_var = sum((c_phi - z_mean) ** 2 for _, _, c_phi in results) / n

r_xy = xy_cov / (math.sqrt(x_var * y_var))
r_xz = xz_cov / (math.sqrt(x_var * z_var))
r_yz = yz_cov / (math.sqrt(y_var * z_var))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": r_xy,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": r_xy >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_r_xy = sum(r["metric_value"] for r in results) / len(results)
        std_r_xy = math.sqrt(sum((r["metric_value"] - mean_r_xy) ** 2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_r_xy} std={std_r_xy} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"r_xy < 0.7\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c847bbd5.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c847bbd5.py", line 72, in run_trial
    instance = generate_instance(n, m)
               ^

```

NameError: name 'm' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing any data, which prevents us from evaluating the Pearson correlation coefficient and supporting or falsifying the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16404
2	preregistration	ollama_remote	glm4:latest	0	0	9186
3	novelty	ollama_remote	glm4:latest	0	0	10562
4	novelty	ollama_remote	glm4:latest	0	0	11706
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20731
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13361
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12491
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16015
9	judge	ollama_remote	glm4:latest	0	0	8671

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119126 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/fb97e5d749a7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fb97e5d749a7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/fb97e5d749a7.tar.gz (if generated)